Shortest Path with Exactly K Special Edges

Algorithm Design & Correctness: Layered Dijkstra Approach

1. Problem

Given a directed graph G = (V, E) with `n` nodes and `m` edges. Each edge has a nonnegative weight `w(e)` and a binary special flag `b(e)` (0 or 1). Given a source `s`, target `t`, and an integer `K ≥ 0`, the goal is to find the minimum-weight directed `s→t` walk that uses exactly `K` special edges. A walk may revisit nodes and edges. If no such walk exists, the output should be IMPOSSIBLE.

2. Key Insight — Layered Graph

Standard Dijkstra's algorithm cannot directly enforce an exact count of special edges. The solution involves augmenting the state space. Instead of tracking only the current node, we track `(node, special-edges-used-so-far)`. This approach creates a layered graph G' with `n × (K+1)` nodes, arranged in `K+1` layers, one layer for each value of `k ∈ {0, 1, ..., K}`.

State: `(v, k)` = "currently at vertex `v`, having traversed exactly `k` special edges"

Layer k	Meaning	Transitions out of (u, k)
0	Used 0 special edges so far	Non-special (b=0): (u,0) → (v,0) +w | Special (b=1): (u,0) → (v,1) +w
k	Used k special edges so far	Non-special (b=0): (u,k) → (v,k) +w | Special (b=1): (u,k) → (v,k+1) +w [if k < K]
K	Used exactly K special edges	Non-special (b=0): (u,K) → (v,K) +w | Special edges: blocked (would exceed K)

The answer is `dist'[(t, K)]` in G', where `dist'` is the shortest-path distance from `(s, 0)`. Non-special edges move horizontally within a layer (k unchanged). Special edges jump from layer `k` to layer `k+1` (consuming one unit of the `K` budget). Special edges out of layer `K` are simply omitted, as they would push `k` above `K`.

3. Algorithm

Dijkstra's algorithm is run on G', starting from `(s, 0)` with distance 0. The layered graph is never explicitly materialized; edges are generated on-the-fly from the original adjacency list during relaxation:

```
dist[s][0] = 0; heap = [(0, s, 0)]
while heap not empty:
    d, u, k = heap.pop_min()
    if d > dist[u][k]: continue # stale entry
    for each edge (u → v, weight w, special b):
        nk = k + b # new layer
        if nk > K: continue # budget exceeded
        if d + w < dist[v][nk]:
            dist[v][nk] = d + w
            heap.push( (d + w, v, nk) )
answer = dist[t][K] (INF ⟹ IMPOSSIBLE)
```

4. Correctness

G' is a directed graph with non-negative weights (inherited from G). Every `s→t` walk in G using exactly `K` special edges corresponds bijectively to an `(s,0)→(t,K)` walk in G' of the same total weight, and vice versa. Dijkstra's algorithm is correct on non-negative weight graphs, so it finds the minimum-weight `(s,0)→(t,K)` path in G', which equals the answer. Walks (not just simple paths) are handled naturally: revisiting `(v,k)` is allowed, but the stale-entry check ensures each state is settled at most once. Zero-weight cycles do not cause infinite loops because they cannot improve distance.

5. Complexity

Quantity	Bound	Reasoning
States	`n(K+1)`	`n` nodes x `(K+1)` layers
Transitions	`m(K+1)`	each original edge copied into `K+1` layers
Dijkstra heap ops	`O(m K log(nK))`	each transition pushed at most once
Numeric bound	`~2x10^7 log(2x10^7)`	`n,m ≤ 2x10^5`, `K ≤ 100` → fully feasible in `< 1 s`
Memory	`O(nK)` for dist table and `O(mK)` for heap entries	both within practical limits for `n,m ≤ 2×10^5` and `K ≤ 100`

6. Edge Cases, Notes & Example

- **K = 0:** Only non-special edges may be used; standard Dijkstra on layer 0 only.
- **K > number of special edges on any s→t path:** `dist[t][K] = INF → IMPOSSIBLE`.
- **Zero-weight edges:** Handled correctly (non-negative weights, stale-entry guard).
- **Disconnected graph:** `dist[t][K]` remains `INF → IMPOSSIBLE`.
- **Walks + floats:** Revisits are allowed (walk, not path); zero-weight cycles are safe (stale guard). Float weights supported; output is integer when whole.